



ARCHITECTURE — dis-bot v2.0 Stable

Документ описывает **архитектуру стабильного билда dis-bot v2.0**.



Всё, что здесь описано, считается **фундаментом проекта**. Изменение существующих механизмов **запрещено** — допускаются только надстройки.



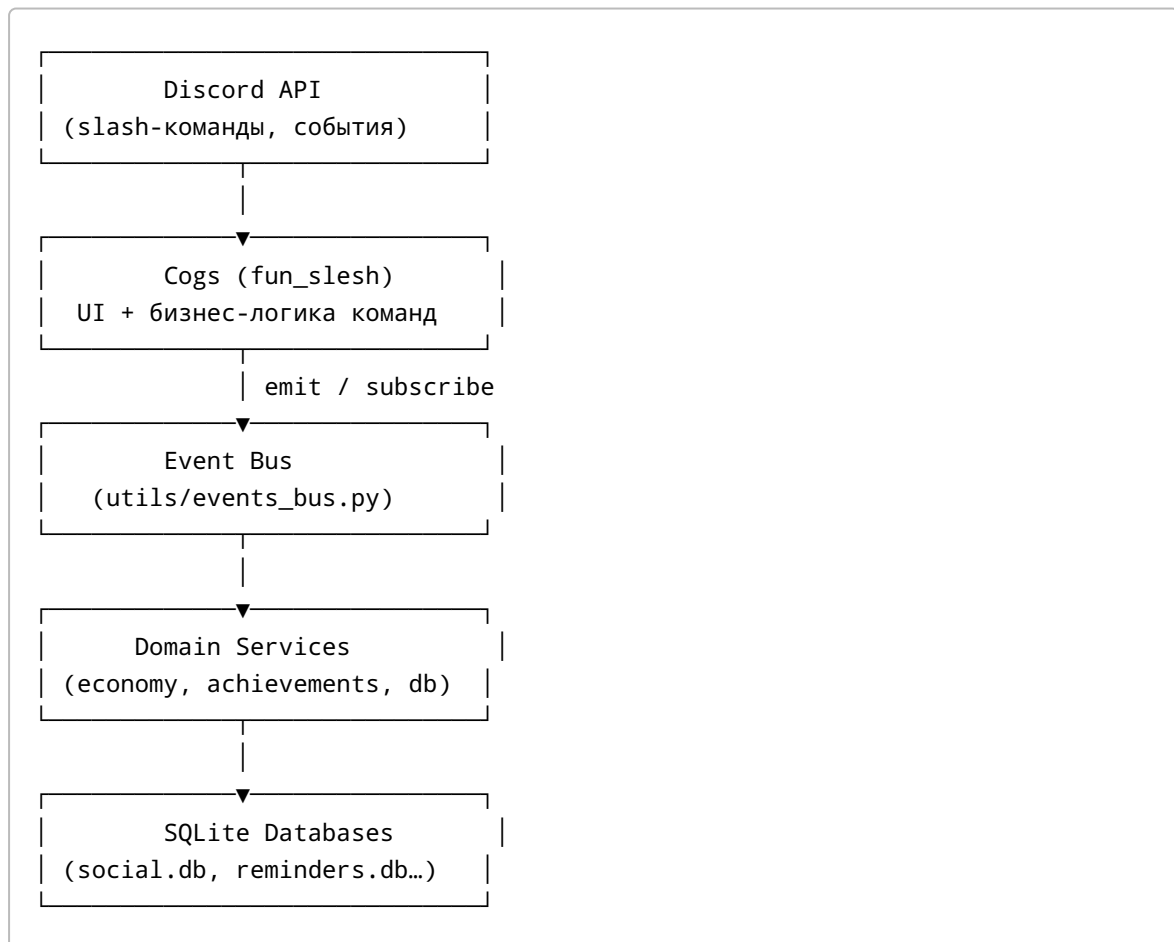
Архитектурные цели

Архитектура dis-bot v2.0 спроектирована так, чтобы:

- быть **модульной** (каждый домен — отдельный Cog);
- иметь **минимальные зависимости между модулями**;
- использовать **локальное хранилище (SQLite)** без внешних сервисов;
- позволять добавлять новые функции **без изменения существующих**;
- быть устойчивой к рестартам бота.



Архитектурные слои



😓 Точка входа

`main_file.py`

Ответственность: - запуск бота; - инициализация логирования; - настройка intents; - автозагрузка всех cogs; - запуск планировщиков.

❌ Не содержит бизнес-логики.

🐱 Cogs — модульная логика

Каждый файл в `fun_slesh/` : - реализует **один домен**; - сам отвечает за свои таблицы БД (`CREATE TABLE IF NOT EXISTS`); - регистрирует **slash-команды**.

Основные cogs

Cog	Ответственность
<code>daily.py</code>	Дэйлик, серии, баланс, топы
<code>games.py</code>	Игры PvE / PvP
<code>rep_and_mood.py</code>	Репутация и настроение
<code>birthday.py</code>	Дни рождения
<code>tools.py</code>	Напоминания, счётчики
<code>mem_roles.py</code>	Временные роли
<code>ai_tools.py</code>	Wikipedia, PubMed
<code>achievements_engine.py</code>	Ачивки
<code>message_and_voice_stats.py</code>	Статистика
<code>test_hello.py</code>	Фан, опросы, инфо

Принцип:

Один cog — одна зона ответственности

👉 Event Bus (ключевой элемент)

`utils/events_bus.py`

Event Bus — это **мягкая связка между модулями**.

Зачем он нужен

- Игры не знают об ачивках

- Дэйлик не знает о наградах
- Репутация не знает о достижениях

Вместо этого:

```
await emit("daily_claimed", user_id=..., streak=...)
```

И подписчик:

```
subscribe("daily_claimed", on_daily_claimed)
```

Гарантии

- события выполняются **последовательно**;
- ошибки обработчиков **не валят бота**;
- нет циклических зависимостей.

⚠ В v2.0 **запрещено**: - прямое обращение одного сог к логике другого; - выдача ачивок напрямую, минуя события.

Domain Services

Экономика (`core/economy.py`)

- единая точка работы с монетами;
- журнал транзакций (ledger);
- защита от silent-изменений баланса.

Все начисления проходят через:

```
add_coins(user_id, delta, reason, meta)
```

Achievements Engine

```
fun_slesh/achievements_engine.py
```

- подписывается на события;
- содержит **только правила** выдачи;
- не зависит от UI или команд.

Архитектурное правило:

Ачивки **реактивны**, а не иницирующие.

Базы данных

Принцип

- SQLite — основной источник истины;
- каждая БД — отдельный домен;
- схемы создаются лениво.

Используемые БД

БД	Назначение
<code>social.db</code>	экономика, ачивки, игры, статистика
<code>reminders.db</code>	напоминания, таймеры, временные роли
<code>birthdays.db</code>	дни рождения

❌ Запрещено: - cross-db JOIN; - shared foreign keys между БД.

Планировщики

Используется **APScheduler**:

- напоминания;
- автоснятие ролей;
- поздравления с ДР.

Особенности: - таймзоны: **MSK / UTC**; - задачи восстанавливаются из БД после рестарта.

Поток данных (примеры)

Дэйлик

```
/дэйлик
↓
Daily Cog
↓
Economy.add_coins()
↓
emit("daily_claimed")
↓
Achievements Engine
```

Игра

```
/кнб
↓
Games Cog
↓
Economy.add_coins()
↓
emit("game_win")
↓
Achievements Engine
```

Архитектурные запреты v2.0

 Изменять существующие таблицы  Ломать сигнатуры событий  Делать прямые вызовы между cogs  Встраивать ML/ETL внутрь текущих модулей

Расширение архитектуры (после v2.0)

Допустимые направления: - новые cogs; - новые события; - новые БД; - ML-модули как **отдельный слой**; - внешние API через адаптеры.

 Всё это **НЕ часть v2.0**, а надстройки.

ARCHITECTURE.md фиксирует фундамент dis-bot v2.0.

Этот документ является точкой отсчёта для всех будущих архитектурных решений.